

Benchmark Report

When does Mamba2 beat the Transformer?

Energy per Token, Throughput and the Crossover Point
of State-Space and Attention Models on an NVIDIA RTX 3070

submitted by

Sumon Kazi

Matriculation no. 23293505

Supervisor: Dr.-Ing. Ayesha Afzal

Programme: Data Science (M.Sc.), Faculty of Sciences

Submitted: Erlangen, October 2026

Abstract

State-space models such as Mamba2 scale linearly with sequence length, whereas softmax attention scales quadratically. This report measures when that advantage becomes real. We benchmark single-stage inference of Mamba2-130m against Pythia-160m and Pythia-70m on an NVIDIA RTX 3070, measuring throughput, GPU power through NVML and energy per token for sequence lengths from 512 to 262,144 tokens and batch sizes from 1 to 16, up to the memory limit.

Mamba2’s cost per token is constant at about 1.92 mJ and 113,000 tokens/s from 2K tokens upwards. The Transformers’ energy per token grows linearly with sequence length ($R^2 > 0.9998$), the signature of quadratic total cost, and the slopes of the two Pythia models differ by 3.01, matching the predicted ratio of 3.00. Mamba2 overtakes Pythia-160m at about 4K tokens (6–8K for larger batches) and the much cheaper Pythia-70m at about 46K tokens; at 256K tokens it needs $14\times$ less energy per token than Pythia-160m. Batching helps only while the GPU is under-utilised. The expected quadratic memory growth of attention did not appear, because PyTorch’s fused attention kernels never store the $L \times L$ score matrix.

Contents

Abstract	i
1 Introduction	1
2 Theoretical Expectation	1
3 Experimental Setup	1
4 Measurement Methodology	2
5 Results	2
5.1 Sequence length	3
5.2 Comparison with the cost model	3
5.3 Batch size	4
5.4 Heatmaps	5
5.5 Power and memory	5
6 Discussion	5
7 Assumptions and Limitations	9
8 Conclusion and Outlook	9
References	9
A Code and Reproduction	9

1 Introduction

Transformers [1] compute attention between every pair of tokens, so processing a sequence of length L costs $O(L^2)$. Selective state-space models such as Mamba [2] and Mamba2 [3] replace attention by a recurrent state update that costs $O(L)$. Asymptotically the state-space model must win, but Transformers run on highly optimised matrix and attention kernels [5], so where the crossover actually lies depends on constant factors that only measurement can reveal. This report answers the question:

At which sequence length does Mamba2 begin to outperform Transformer baselines of comparable size in throughput and energy per token, and how do sequence length, batch size, architecture and parameter count shape this trade-off?

2 Theoretical Expectation

A decoder-only Transformer with $N \approx 12 n_{\text{layer}} d^2$ non-embedding parameters, n_{layer} layers and hidden size d needs per token approximately

$$C_{\text{token}}(L) \approx \underbrace{2N}_{\text{projections + MLP}} + \underbrace{2 n_{\text{layer}} d L}_{\text{causal attention}} \quad (1)$$

floating-point operations, where the attention term counts $QK^\top$ and AV over on average $L/2$ preceding tokens. The cost per token is therefore linear in L and the total cost quadratic. Attention dominates beyond

$$L^* = \frac{2N}{2 n_{\text{layer}} d} \approx 12 d, \quad (2)$$

that is, about 9.2K tokens for Pythia-160m ($d = 768$) and 6.1K for Pythia-70m ($d = 512$). Well above L^* , doubling L should nearly double the energy per token.

Mamba2 [3] processes the sequence with a selective state-space recurrence: each token updates a fixed-size state, so the cost per token is independent of L . At short lengths it is nevertheless more expensive than a Transformer of equal size, because it has more layers (24 vs. 12) and its scan kernels use the GPU less efficiently than dense matrix multiplications.

For memory, naive attention stores an $L \times L$ score matrix per head and grows quadratically. Fused kernels such as FlashAttention [5], used by PyTorch’s default attention, compute attention in tiles and never store this matrix, so with them attention memory grows only linearly.

3 Experimental Setup

All measurements ran on one machine of the FAU Computer Science CIP pool; `nvidia-smi` confirmed before each run that no other process used the GPU.

Table 1. Hardware and software configuration.

Component	Configuration
GPU	NVIDIA GeForce RTX 3070, 8 GB, 220 W power limit, driver 610.57 (CUDA 13.3)
Host	Intel Core i7-12700, 64 GB RAM, Debian Linux, gcc 14.2
Software	Python 3.11, PyTorch 2.12.1+cu130, Transformers 5.12.1
Mamba kernels	mamba-ssm 2.2.6, causal-conv1d 1.6.2 (built from source)
Energy interface	NVML via nvidia-ml-py 13.610.43
Numerics	float16, torch.inference_mode

All three models were trained on the Pile and share the GPT-NeoX tokenizer (Table 2). Because embedding tables cost almost no compute, non-embedding parameters are the relevant size measure: Mamba2-130m vs. Pythia-160m isolates the *architecture*, and Pythia-70m vs. Pythia-160m isolates the *parameter count* [4].

Table 2. Models under test (parameter counts approximate).

Model	Architecture	Layers	d	Non-embedding params
Mamba2-130m	State-space model (SSD scan)	24	768	$\approx 90\text{M}$
Pythia-160m	Transformer, softmax attention	12	768	85.1M
Pythia-70m	Transformer, softmax attention	6	512	18.9M

4 Measurement Methodology

Inference is treated as a single stage: one forward pass over B sequences of L random token ids, computing logits for the last position only, so that the large output layer does not dominate memory and hide the architectural difference. Pythia uses PyTorch’s default attention and is also run beyond its 2048-token training context, which is valid for compute and energy but not for output quality.

Each configuration is measured in five steps:

1. **Idle baseline:** GPU power at rest, measured for 5 s ($P_{\text{idle}} = 16.8\text{ W}$).
2. **Warm-up:** at least 3 iterations and 2 s unrecorded, so kernels compile and clocks stabilise.
3. **Calibration:** the number of iterations is chosen so that each measured window lasts at least 4 s. A single forward pass can finish in under 100 ms, shorter than NVML’s power update interval, so short windows would miss it.
4. **Measurement:** the iterations run back to back between two GPU synchronisations; energy comes from the NVML energy counter read before and after the window, cross-checked by polling power every 10 ms. Each window is repeated 2–3 times.
5. **Metrics:** throughput $= BLn/t$ and energy per token $= E/(BLn)$ for n iterations, duration t and energy E .

Two sweeps were run: a sequence-length sweep at $B = 1$ with L from 1K to 32K (3 repeats), and a grid with $B \in \{1, 2, 4, 8, 16\}$ and L doubling from 512 until the GPU ran out of memory (2 repeats). Crossover points are interpolated between neighbouring lengths and are therefore accurate to within one doubling.

5 Results

5.1 Sequence length

Table 3 and Figure 1 show the results at batch size 1.

Table 3. Batch-1 sweep (mean of 3 windows).

Model	Metric	1K	2K	4K	8K	16K	32K
Mamba2-130m	tokens/s	81,636	108,148	112,055	109,854	110,605	112,580
	mJ/token	2.056	1.933	1.930	1.984	1.956	1.926
	power (W)	164	208	216	215	215	215
Pythia-160m	tokens/s	108,746	117,142	110,408	97,399	72,717	47,450
	mJ/token	1.642	1.704	1.939	2.200	2.963	4.557
	power (W)	177	197	213	215	213	216
Pythia-70m	tokens/s	343,431	405,225	362,699	308,524	235,360	149,998
	mJ/token	0.475	0.474	0.564	0.695	0.909	1.437
	power (W)	161	192	204	212	213	214

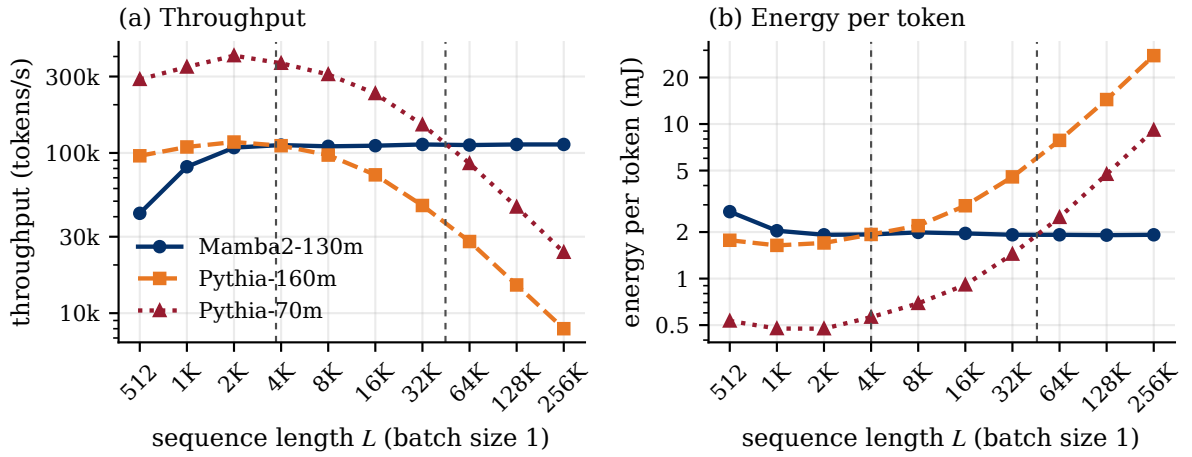


Figure 1. Throughput (a) and energy per token (b) versus sequence length at batch size 1. Dashed lines mark the crossover points.

Mamba2 is flat from 2K to 256K tokens at about 113,000 tokens/s and 1.92 mJ per token; below 2K a single sequence does not fully use the GPU. Pythia-160m peaks at 2K and falls to 8,000 tokens/s and 27.7 mJ per token at 256K. Pythia-70m is 3–7 $\times$ faster than Mamba2 at short lengths but degrades the same way. Table 4 lists the crossover points.

Table 4. Crossover points and Mamba2’s advantage at the longest length.

Comparison	Throughput	Energy	Advantage at longest L
vs. Pythia-160m, $B = 1$	$\approx 3.8K$	$\approx 4.0K$	14 $\times$ less energy (256K)
vs. Pythia-160m, $B = 2-16$	$\approx 6-8K$	$\approx 6-8K$	1.5–7.7 $\times$
vs. Pythia-70m, all B	$\approx 46K$	$\approx 47K$	4.8 $\times$ less energy (256K)

5.2 Comparison with the cost model

Equation (1) predicts that the Transformers’ energy per token is linear in L . A least-squares fit $E(L) = a + bL$ for $L \geq 4K$ confirms this (Figure 2, Table 5); Mamba2’s slope is zero.

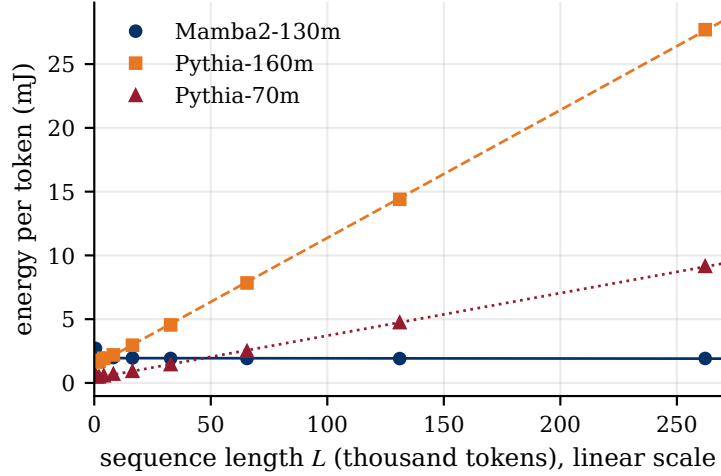


Figure 2. Energy per token at batch size 1 on linear axes, with fitted lines for $L \geq 4K$.

Table 5. Linear fit of energy per token, $E(L) = a + bL$, for $L \geq 4K$ at $B = 1$.

Model	a (mJ)	b (μ J per token of context)	R^2	a/b	L^* from Eq. (2)
Pythia-160m	1.347	0.1003	0.99989	13.4K	9.2K
Pythia-70m	0.375	0.0333	0.99981	11.2K	6.2K

The attention term in Equation (1) is proportional to $n_{\text{layer}}d$: 9216 for Pythia-160m and 3072 for Pythia-70m, a ratio of 3.00. The measured slopes have a ratio of 3.01. Between 32K and 256K, energy per token grows by 1.72, 1.84 and 1.92 per doubling for Pythia-160m (1.73, 1.90, 1.93 for Pythia-70m), approaching the asymptotic factor of 2; for Mamba2 the factors are 1.00, 0.99 and 1.01.

5.3 Batch size

Figure 3 shows throughput versus batch size. At $L = 512$, going from $B = 1$ to $B = 16$ raises Pythia-160m’s throughput by 61% and Pythia-70m’s by 81%, and Mamba2’s by $2.6\times$. From about 16K tokens upwards, all batch sizes give the same throughput for the Transformers; Mamba2 is flat beyond about 2–4K tokens per batch.

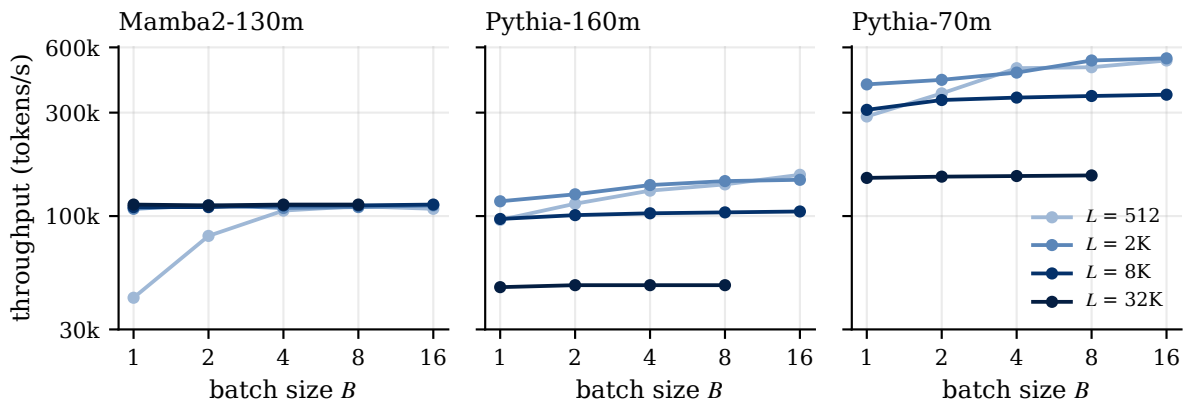


Figure 3. Throughput versus batch size for four sequence lengths. Lines end where memory runs out.

5.4 Heatmaps

Figures 4–6 show the full grid. Grey cells ran out of memory.

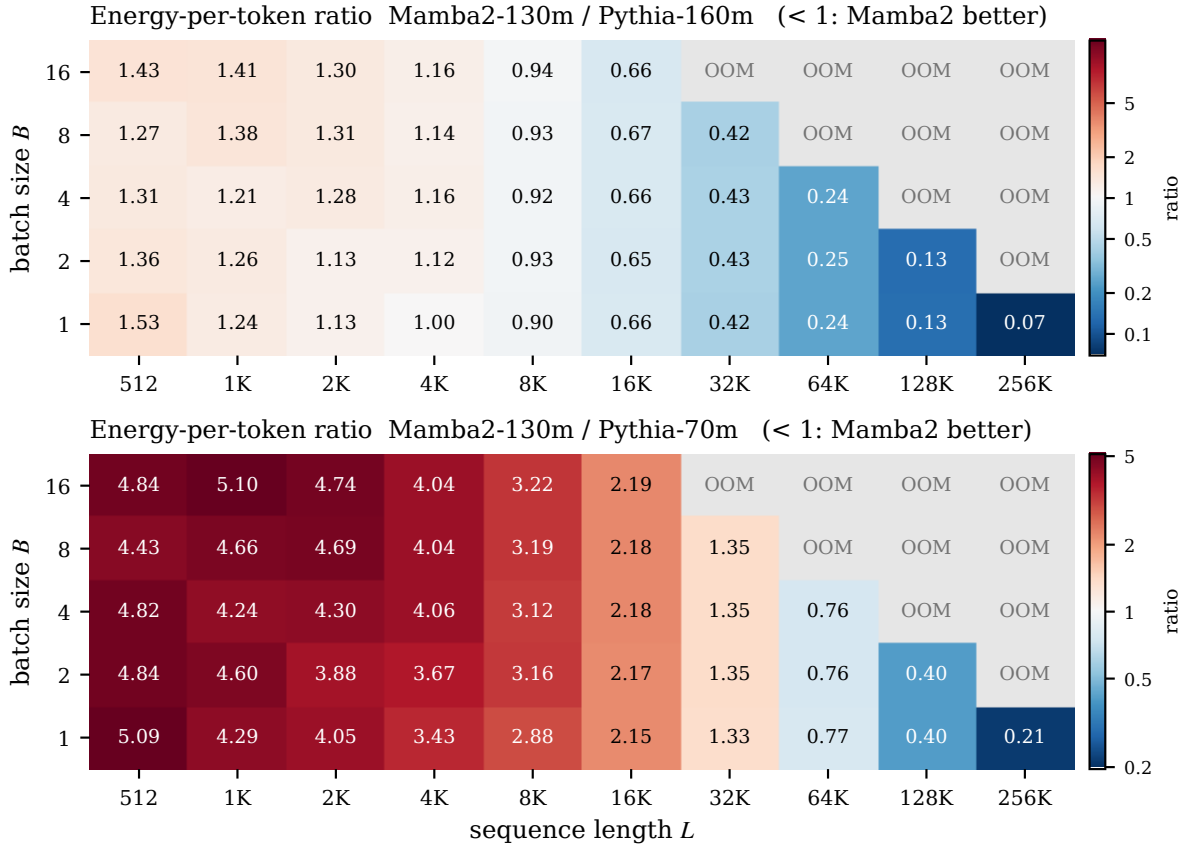


Figure 6. Energy per token of Mamba2-130m divided by each Transformer. Blue (< 1): Mamba2 is better; the white boundary is the crossover.

The Mamba2 heatmaps are almost uniform. In the Transformer heatmaps the colour changes along the sequence-length axis and hardly along the batch axis, except at short lengths. The throughput ratios are nearly identical to the energy ratios in Figure 6.

5.5 Power and memory

The GPU drew 213–217 W in almost every configuration, close to its 220 W limit; lower power occurred only when it was under-utilised (161–177 W at $B = 1$, $L = 1K$). Peak memory grew linearly with $B \cdot L$ for all models, and all three ran out of memory at 524,288 tokens per batch while 262,144 always fitted. At 32K tokens and $B = 1$, Mamba2 used 1164 MB, Pythia-160m 900 MB and Pythia-70m 531 MB.

6 Discussion

Where the crossover is, and why. Mamba2 overtakes the similar-size Pythia-160m at about 4K tokens. Below that, the Transformer is cheaper although it does comparable work per token: its length-independent cost ($a = 1.35$ mJ) is about 30% lower than Mamba2’s 1.92 mJ. Per unit of work, Pythia-160m’s projections and MLP reach about 126 GFLOP/J, Mamba2 only about 94 GFLOP/J, because

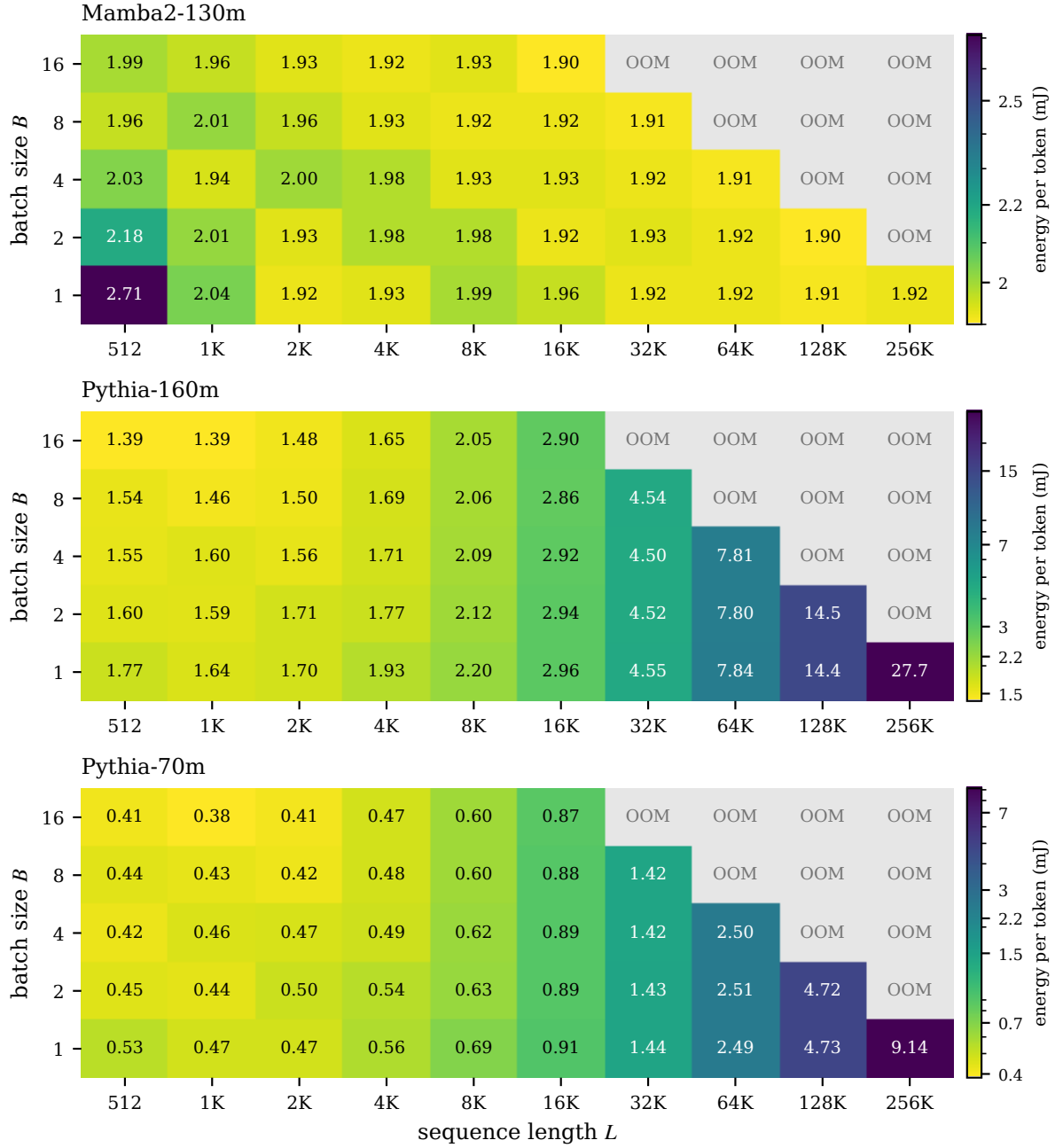


Figure 4. Energy per token (mJ) over batch size and sequence length; each panel has its own colour scale.

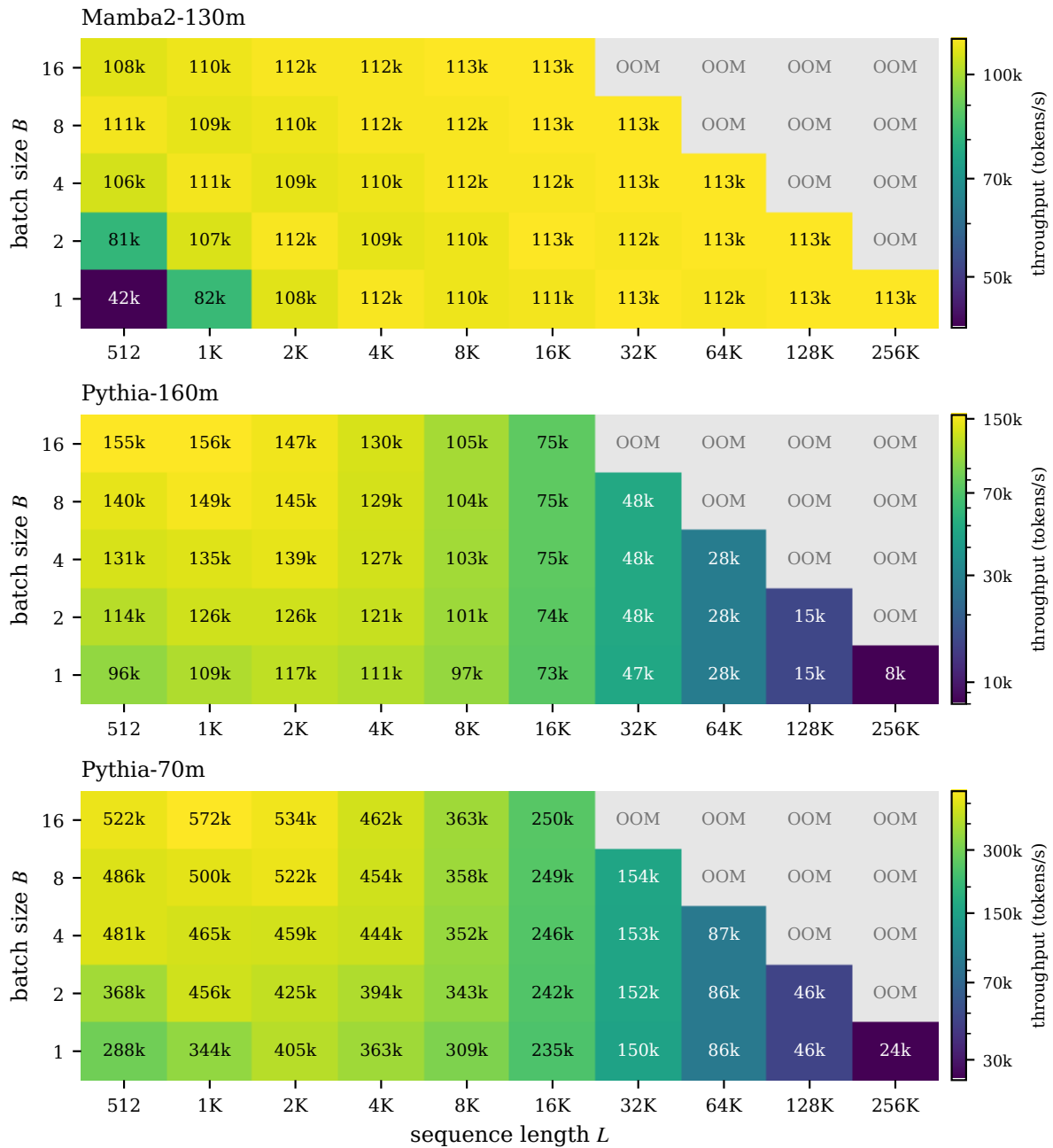


Figure 5. Throughput (tokens/s) over batch size and sequence length; each panel has its own colour scale.

dense matrix multiplications run more efficiently than the scan and Mamba2 has twice as many layers. The crossover is the length at which the Transformer’s growing attention cost, bL , has used up this constant-factor advantage.

Agreement with theory. For compute and energy the measurements confirm the theory closely: Mamba2’s cost per token is constant over more than two orders of magnitude of L ; the Transformers’ cost per token is linear in L ($R^2 > 0.9998$); and the slope ratio of 3.01 matches the predicted 3.00. Dividing the attention FLOPs per token of context ($2n_{\text{layer}}d$) by the slope gives the same 184 GFLOP/J for both Pythia models, as expected since both use the same attention kernel. The one deviation is that attention starts to dominate later than Equation (2) predicts (13.4K and 11.2K instead of 9.2K and 6.2K): attention runs more efficiently per FLOP (184 GFLOP/J) than the projection and MLP layers (126 and 101 GFLOP/J), so FLOP counts locate the crossover only approximately.

Parameter count. Pythia-70m does about $4.5\times$ less work per token than Pythia-160m, yet its attention cost still grows with L , and the fitted model predicts its crossover with Mamba2 at $(1.92 - a)/b = 46.4\text{K}$, matching the measured 46K. Shrinking the Transformer moved the crossover about $12\times$ further out but did not remove it: a constant-factor advantage only delays a crossover against a cost that grows with L .

Batch size. Batching only helps while the GPU still has idle capacity: for the Transformers at short sequences, for Mamba2 only at 512–1K tokens. Because it helps the Transformer more at short lengths, it shifts the Pythia-160m crossover from about 4K to 6–8K tokens; against Pythia-70m the crossover is independent of batch size.

Energy and throughput. Because the GPU runs at its power limit almost everywhere, energy per token is essentially power divided by throughput, and both metrics tell the same story. They differ only for under-utilised configurations: at $B = 1$, $L = 512$ Mamba2 is $2.27\times$ slower than Pythia-160m but uses only $1.53\times$ more energy, because it draws less power.

Sweet spots. The Transformers are most efficient for short, batched inputs: Pythia-70m at $B = 16$, $L = 1\text{K}$ reaches 572,000 tokens/s at 0.384 mJ per token, Pythia-160m about 155,000 tokens/s at 1.39 mJ at $B = 8\text{--}16$ and $L \leq 1\text{K}$. Mamba2 runs at about 113,000 tokens/s and 1.90 mJ in almost any configuration that fills the GPU, which makes its cost predictable. Short-context, batch-heavy workloads therefore favour a small Transformer, while sequences beyond about 4–8K tokens favour Mamba2 over a Transformer of the same size.

Where the results do not match theory: memory. The expected quadratic memory growth of attention did not appear; memory grew linearly for all models and Mamba2 even used slightly more than Pythia-160m. PyTorch’s fused attention [5] never stores the $L \times L$ matrix, so attention compute remains quadratic but its memory becomes linear. Mamba2’s real memory advantage is expected during autoregressive decoding, where its state has a fixed size while a Transformer’s key-value cache grows with L ; a single forward pass does not test this.

7 Assumptions and Limitations

- Only a single forward pass was measured; prefill and decode were not separated, so decoding and the key-value cache are not covered.
- Only last-token logits were computed, and inputs were random tokens; output quality was not evaluated, and Pythia was run beyond its training context.
- NVML measures GPU power only (no CPU, memory or power-supply losses), and its internal averaging limits accuracy for short events; long windows reduce but do not remove this.
- Results come from one power-limited GPU on a shared machine; each point is the mean of 2–3 windows, and FLOP and parameter counts are approximate.
- `mamba-ssm 2.2.6` was used with a small compatibility patch instead of the latest release.

8 Conclusion and Outlook

Mamba2’s linear scaling becomes a real advantage at about 4K tokens against a Transformer of comparable size and at about 46K tokens against a $4.5\times$ cheaper one, in both throughput and energy per token. The measurements confirm the theoretical cost structure closely: constant cost per token for Mamba2, and a linearly growing cost per token for the Transformers whose slope scales exactly with $n_{\text{layer}}d$. Where the crossover lies is set by kernel efficiency. Batching helps only while the GPU is under-utilised, energy per token on this power-limited GPU follows throughput, and quadratic memory growth does not occur with fused attention.

Natural next steps for the thesis are to measure prefill and decode separately, to compare fused with naive attention, and to repeat the study on other GPUs and with larger models.

References

- [1] A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017.
- [2] A. Gu, T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*, 2023.
- [3] T. Dao, A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *ICML*, 2024.
- [4] S. Biderman et al. Pythia: A suite for analyzing large language models across training and scaling. *ICML*, 2023.
- [5] T. Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

A Code and Reproduction

The submitted code consists of `benchmark_energy.py` (initial benchmark), `benchmark_sweep.py` (sweeps) and `plot_sweep.py` (figures); the raw measurements are in `sweep_b1.csv` and `sweep_grid.csv`. The sweeps were run with:

```
python benchmark_sweep.py --batch-sizes 1 \
    --seq-lens 1024 2048 4096 8192 16384 32768 --out sweep_b1.csv
python benchmark_sweep.py --batch-sizes 1 2 4 8 16 \
    --seq-lens 512 1024 2048 4096 8192 16384 32768 \
```

```
--until-oom --repeats 2 --out sweep_grid.csv
```